ChromSimPipe Code Guide

End-to-end walkthrough of the cohesin loop-extrusion simulation codebase

2026

CODE_GUIDE: What Every File in This Repo Actually Does
- Quick-Start and Codebase Walkthrough
- The Snakemake pipeline files
- Reading order
- Project layout
- 1. Configuration: `configs/parameters.py`
- 2. Hi-C data preparation
- 3. CTCF annotation
- 4. The LEF dynamics engine
- 5. Running simulations
- 6. Post-simulation analysis
- Legacy cluster scripts (`legacy/cluster/`)
- Conventions you'll see everywhere
- Common pitfalls

CODE_GUIDE: What Every File in This Repo Actually Does

This is a plain-language walkthrough of the whole codebase. It is written for people who don't read Python and want to understand, end-to-end, what runs, what each script produces, and how the files feed into each other.

If you just want to run the pipeline, see `README.md`. Come back here when you need to know *why* a step exists, what an output file contains, or where a specific variable is set.

Quick-Start and Codebase Walkthrough

What this pipeline does

ChromSimPipe is a physics-based simulation framework for testing mechanistic hypotheses about 3D chromatin organisation. Given CTCF binding data and Hi-C maps, it simulates cohesin loop extrusion under different parameter conditions and compares the resulting contact maps to experimental Hi-C.

The included example (STRS project, Flores et al. 2026) asks: **can we explain the stress-induced changes in Hi-C contact maps purely from what we can measure — changes in CTCF binding and cohesin dynamics — without invoking anything we can't observe?**

The simulation models cohesin loop extrusion: cohesin rings load onto chromatin and extrude DNA into loops, stalling when they encounter CTCF sites oriented the right way. Where cohesins stall and how long they live determines the 3D contact map. By varying which CTCF sites are present and cohesin dynamics parameters, the simulation tests user-defined hypotheses.

Example conditions (shipped in `config/ChromSimConfig.yaml`):

Condition	Cohesin	CTCF sites	Tests
`cond1_baseline`	Baseline	Condition 1	Does the model reproduce condition 1 Hi-C?
`cond2_ctcf-only`	Baseline	Condition 2	Do condition 2 CTCF sites alone explain condition 2 Hi-C?
`cond2_weak-ctcf`	Baseline	Condition 2 (0.5× stall)	Does weaker stalling at remaining CTCF sites help?

Data flow

```
ChromSimSamplesheet.txt  (one row per sample: metadata,
CTCF_Peaks, Hi-C)
    ↓  orientCTCF rule → scripts/extract_ctcf_sites.py  (strand
via FIMO)
Oriented CTCF BED files (output/ctcf/)
    ↓  simulateShard rule → scripts/run_simulation_shard.py +
polychrom
Simulated conformations (output/sims/)
    ↓  analyzeCondition rule → scripts/run_analysis_all.py
Comparison figures  ←→  experimental Hi-C (.cool, from .hic via
the convertHic rule)
```

Setup and launch

The workflow follows the PhanstielLab Snakemake pattern (bagPipes, dietJuicer): edit a samplesheet and a config, then submit one launcher. There is no separate setup script — converting Hi-C, orienting CTCF, and building conda environments are all workflow steps.

1. **`ChromSimSamplesheet.txt`** — one tab-separated row per sample. The required columns are `CTCF_Peaks` and `Hi-C` (`NA` when a sample has no file of that kind); all other columns are metadata.
2. **`config/ChromSimConfig.yaml`** — `fileNamesFrom` names the samplesheet columns joined with _ to name each sample. Conditions refer to samples by those names (`ctcf_type`, `compare_hic`). The config also sets the locus, physics, genome, CTCF motif, conda environments, and `outputDir`.
3. **`profiles/<site>/`** — the compute site, named by `hpcProfile` in the config. `config.yaml` holds the scheduler, account and each rule's CPUs / memory / GPU under `set-resources:`; `env.sh` loads the software. Shipped: `longleaf` (UNC), `pioneer` (Case Western, with its own `config/ChromSimConfig.pioneer*.yaml` and samplesheet), `local` (no scheduler), and `slurm-template` (copy and fill in for any other SLURM cluster).

```
sbatch ChromSimPipe.sh        # whole workflow
bash ChromSimPipe.sh -n       # dry run
sbatch ChromSimPipe.sh --configfile
config/ChromSimConfig.test.yaml   # 9-job smoke test
```

Test config. `config/ChromSimConfig.test.yaml` is the same config with one condition, one replicate, one shard and 1,000 blocks, written to `output_test/`. Run it first on any new cluster: it touches every rule, including one GPU simulation, for a fraction of the cost of a full run.

Email. `export CHROMSIM_EMAIL=you@university.edu` and the workflow emails you when it starts, finishes, or fails (see `notify:` in the config).

The central config: `config/ChromSimConfig.yaml`

This YAML file and the samplesheet are the source of truth for everything. `configs/parameters.py` reads both at import time. From the `locus:` block it derives `CHROM`, `REGION_START`, `REGION_END`, and `N_MONOMERS` (1,960 for the default window at 1 kb per monomer).

CTCF auto-loading. When any script imports `configs.parameters`, it opens the oriented CTCF BED for each CTCF sample and locus if it already exists (e.g. `output/ctcf/ctcf_oriented_hg38_condition1_<locus>.bed`). It parses each line into `(monomer_position, orientation)` where `+1` means a forward-strand motif and `−1` means reverse. If a BED does not exist yet, nothing is loaded, and `get_ctcf_arrays()` raises an error when a script asks for those sites. There are no placeholder sites.

Cohesin parameters. Each condition has a parameter set, a dict with `lifetime` (mean steps before cohesin falls off), `separation` (spacing between cohesins), `ctcf_capture` (probability of stalling per CTCF encounter), and `ctcf_release` (escape probability per step while stalled). Values come from Gabriele et al. 2022 (mESC measurements used as the human baseline).

Tiling. Rather than simulating 1,960 monomers, the simulation runs 70,000 monomers arranged as 28 back-to-back copies of the locus (each tile = 1,960 monomers + 270 padding on each side). All 28 tiles are independent, so each GPU run produces ~28× the contact statistics at no extra physics cost. Contact maps are averaged back down to 1,960×1,960 for comparison. All three numbers are in the `tiling:` block.

The CTCF extraction: `scripts/extract_ctcf_sites.py`

Your `.narrowPeak` file has peak coordinates but no strand information. Orientation matters because a + CTCF site blocks a cohesin arm moving leftward, while a − site blocks one moving rightward — so a convergent pair (+ on the left, − on the right) creates a stably anchored loop, while a divergent pair does not.

This script:

1. Takes each peak's summit from the narrowPeak file (column 10).
2. Extracts ± `ctcf_motif.flank_bp` around each summit with `bedtools getfasta`.
3. Runs FIMO for the motif in the MEME file from the `getMotif` rule (JASPAR `ctcf_motif.jaspar_id`, default MA0139.1).
4. Uses the strand of the best hit as the site orientation.

5. Handles peaks with no hit as `ctcf_motif.unoriented` says: `drop` leaves them out and lists them in the log; `error` stops the run. Orientations are never guessed.
6. Writes a BED6 file where column 6 is `+` or `−`.

The 1D engine: `scripts/lef_dynamics.py`

Before any 3D physics, loop extrusion is modelled as a purely 1D stochastic process. `LEFSimulator` runs this:

- Each cohesin is a pair of arms that step away from each other along the chromatin fiber, one lattice site per simulation tick.
- An arm stalls when it hits a CTCF site with the blocking orientation, with probability `ctcf_capture`. It escapes with probability `ctcf_release` per step.
- Arms also stall when they collide with another cohesin (steric exclusion).
- Each cohesin lives for a geometrically distributed number of steps with mean `lifetime`, then unbinds and is replaced at a random position. Stalled cohesins unbind at the same rate.

The 3D simulation: `scripts/run_simulation_shard.py`

This is where polychrom (OpenMM on GPU) runs. For each block:

1. `LEFSimulator.step()` advances all cohesins one tick on the 1D lattice.
2. Cohesin arm positions feed into polychrom as harmonic spring bonds between the two bridged monomers.
3. OpenMM integrates the polymer under those bonds for `md_steps_per_block` (250) MD steps.
4. polychrom's `HDF5Reporter` saves the coordinates of the block. Contacts (pairs within `contact_radius` = 3.0) are counted later, in the analysis.

The analysis: `analysis/`

- **`contact_maps.py`** — accumulates saved conformations into a 2D contact frequency matrix using `scipy.spatial.cKDTree` for fast distance lookups. This is the simulated analogue of a Hi-C contact map.
- **`ps_curve.py`** — computes P(s): contact probability as a function of genomic separation. The shape of this curve is the primary comparison metric against real Hi-C.
- **`experimental_compare.py`** — loads your `.cool` file via `cooler` and extracts the same locus at the configured resolution.
- **`msd_two_point.py`** — tracks mean-squared displacement of specific monomer pairs over time (the in-silico analogue of live-cell imaging).

The Snakemake pipeline files

The workflow is a Snakemake DAG written in the style of the PhanstielLab pipelines: a samplesheet plus a config drive everything, every rule writes its own log and benchmark, rules carry no cluster resources (the profile does), and one `sbatch` launches it all.

`ChromSimPipe.sh` — sbatch entry point

Submit with `sbatch ChromSimPipe.sh`. It:

1. Sources `<hpcProfile>/env.sh`, which makes `python3` and `conda` available on that site (`--profile` overrides `hpcProfile`).
2. Creates a venv in `env/` (first run only) and installs `config/requirements.txt` (Snakemake 8.27.1 + the SLURM executor plugin).
3. Runs `snakemake -s workflows/ChromSimPipe.snakefile --profile <hpcProfile> --configfile <config> --rerun-incomplete --keep-going -p`, adding `--slurm-logdir <outputDir>/logs_slurm` when the profile uses the SLURM executor.

Any extra arguments go to `snakemake` (`-n` for a dry run, `--configfile other.yaml`, `--until validateCTCF`, …).

`unlock.sh` — release lock after failure

Snakemake writes a `.snakemake/locks/` directory while running. If the job dies without cleanup, the lock persists and the next run refuses to start. `bash unlock.sh` runs `snakemake --unlock` against the workflow to clear it.

`workflows/ChromSimPipe.snakefile` — the DAG

Rule	Runs	What it does
`convertHic`	per Hi-C sample	`hic2cool convert` `.hic` → `.cool` at `resolution`
`getMotif`	once	`wget` the JASPAR motif (skipped when `ctcf_motif.meme` is set)
`orientCTCF`	per CTCF sample used by a condition	`extract_ctcf_sites.py`
`validateCTCF`	once	`validate_ctcf.py` — sanity-checks all BED files

Rule	Runs	What it does
`simulateShard`	condition × replicate × shard (GPU)	`run_simulation_shard.py` — 36 jobs with the example config
`mergeShards`	condition × replicate	`merge_shards.py` — concatenates shard HDF5 streams
`analyzeCondition`	per condition	`run_analysis_all.py` — contact maps, P(s), APA, figures
`benchmarkSummary`	once	`workflows/utils/benchmarking.py`

Every rule has the same parts, following bagPipes:

- `log:` — stdout / stderr to `output/logs/{rule}_{wildcards}.out|.err`
- `benchmark:` — `output/benchmarks/{rule}_{wildcards}.tsv` (wall time, memory), summarised in `output/{runName}_benchmarking.tsv`
- `conda:` — an entry from `condaEnvs` in the config; the shell runs scripts with `"$CONDA_PREFIX/bin/python"`, that environment's own interpreter
- `params:` — every value read from the config; nothing is typed in

Wildcards: `{sampleName}` / `{ctcfType}` are samplesheet sample names; `{condition}` is a condition `name`; `{rep}` and `{shard}` expand over `range(n_replicates)` and `range(n_shards)`.

Notifications. `onstart:`, `onsuccess:` and `onerror:` handlers run once in the main Snakemake process and call `notify()`, which runs the `notify: command:` from the config with the subject and address filled in and the message on stdin. The address is `$CHROMSIM_EMAIL`, else `notify: email:`; with neither set, nothing is sent. The error message includes the last 40 lines of the Snakemake log. Sending is wrapped so a mail failure can never fail the run, and handlers do not run on dry runs.

The Snakefile passes the config's path to every script in `$CHROMSIM_CONFIG`, so scripts read the same file Snakemake was given. Steps whose real outputs are directories (simulation, merge, analysis) are tracked through completion markers in `output/done/`.

`profiles/<site>/config.yaml` — scheduler settings

This is the only place cluster details live. In `profiles/longleaf/`, `default-resources:` sets the account, partition, wall time, and memory for every job. `set-resources:` sets each rule's request (the dietJuicer pattern). `simulateShard` is sent to whichever of `l40-gpu`, `volta-gpu` or `gpu` has a free node first, with `qos: gpu_access` and `gpu: 1`. `software-deployment-method: conda` makes Snakemake activate each rule's environment.

`ChromSimSamplesheet.txt` — samples

```
Project  Cell_Type  Condition   CTCF_Peaks
Hi-C
STRS     HEK293T    condition1  /path/to/control.narrowPeak
/path/to/control.hic
STRS     HEK293T    condition2  /path/to/sorbitol.narrowPeak
/path/to/sorbitol.hic
```

With `fileNamesFrom: ['Condition']` the samples are `condition1` and `condition2`. Metadata values shared by every row (`STRS`, `HEK293T`) form the run name, `STRS_HEK293T`, used for run-level files. The workflow writes the parsed samplesheet, with a `sampleName` column, to `output/{runName}_ChromSimSamplesheet.txt`.

Reading order

Everything here is organised by pipeline stage, top-down:

1. The Snakemake pipeline files — **start here**
2. Project layout — what each folder is for
3. Configuration — the single file that controls every simulation
4. Hi-C data preparation — turning experimental data into comparison matrices
5. CTCF annotation — turning CUT&Tag peaks into oriented barrier lists
6. The LEF dynamics engine — the loop-extrusion simulator
7. Running simulations — the shard, merge, and single-run entry points
8. Post-simulation analysis — contact maps, P(s), APA, MSD, calibration
9. Legacy cluster scripts — old sbatch wrappers (superseded by ChromSimPipe.sh)
10. Conventions you'll see everywhere — monomers, tiling, output directory names
11. Common pitfalls — what to check when something looks wrong

Every script section follows the same format:

- **File** — path in the repo
- **Purpose** — one-sentence summary
- **Inputs** — command-line flags and files read
- **Outputs** — files written, with exact paths
- **How to run it** — example command
- **What it does, step by step** — plain-language walkthrough

Project layout

```
ChromSimPipe/
├── ChromSimPipe.sh   – sbatch launcher
├── ChromSimSamplesheet.txt – one row per sample (edit before
first run)
├── config/           – ChromSimConfig.yaml + requirements.txt
(edit before first run)
├── configs/          – parameters.py (reads the config +
samplesheet; single source of truth)
├── envs/             – conda environment specs used by the
rules
├── scripts/          – Python simulation, analysis, and utility
scripts
├── analysis/         – Python modules imported by
scripts/run_analysis_all.py
├── workflows/        – Snakemake DAG (ChromSimPipe.snakefile) +
utils/
├── profiles/         – one folder per compute site (longleaf,
pioneer, local, slurm-template)
├── legacy/           – unmaintained code from the lab's
mESC/neuron project (see legacy/README.md)
├── tests/            – test suite
└── output/           – everything the workflow writes (git-
ignored; set by outputDir)
```

1. Configuration: `configs/parameters.py`

Purpose. This file is the runtime source of truth for every simulation parameter. It reads the config (`$CHROMSIM_CONFIG`, default `config/ChromSimConfig.yaml`) and the samplesheet it names at import time, and re-exports a fixed set of variables so that all downstream scripts stay unchanged when config values change.

Key exported names:

- **Locus** (`CHROM`, `REGION_START`, `REGION_END`, `RESOLUTION`, `N_MONOMERS`, `ACTIVE_LOCUS`). Derived from the `locus:` block in `ChromSimConfig.yaml`. For the included STRS example: hg38, chr1:64,210,000–66,170,000 → 1,960 monomers at 1 kb/monomer.

- **Samples and outputs** (`SAMPLES`, `RUN_NAME`, `OUTPUT_DIR`, `PATHS`). Sample names, metadata and input files from the samplesheet; every output directory under `outputDir`.

- **Polymer physics** (`POLYMER` dict). Bond lengths, bending stiffness (`angle_force`), confinement density, integration tolerance, GPU platform. Rarely changed between projects.

- **LEF ↔ polymer bonds** (`SMC_BOND`). How stiff the cohesin "ring" is in 3D — how strongly it pulls the two loop flanks together.

- **CTCF tracks** (`CTCF_TYPES`, `CTCF_PEAKS`, `CTCF_BEDS`). One oriented BED per `ctcf_type` used by a condition, under `output/ctcf/`. At import time, `load_ctcf_from_bed()` reads each file into `CTCF_SITES_BY_TYPE[type]` — monomer indices with `+1` / `−1` orientation labels. Type names are samplesheet sample names, so any number of CTCF samples is supported.

- **Cohesin parameter sets** (`ALL_PARAM_SETS`). One dict per condition hypothesis, each with `lifetime`, `separation`, `ctcf_capture`, and `ctcf_release`. Values from Gabriele et al. 2022.

- **Condition list** (`SIMULATION_CONDITIONS`). The rows the Snakemake pipeline iterates over. Each entry pairs a cohesin parameter set with a CTCF track and has a `name` (e.g. `cond1_baseline`).

- **Tiling** (`TILING`). The locus is copied 28× into one long polymer for 28× better contact statistics at no extra physics cost. `TILING["chrom_size"]` is the full polymer length (~70,000 monomers).

- **Run parameters** (`SIM_RUN`). `engine`, `seed_base`, `total_blocks`, `md_steps_per_block`, `save_every`, `warmup_blocks`, `hdf5_frames_per_file`.

- **Analysis** (`ANALYSIS`, `MSD_PROBE`, `MSD_FIT`, `CALIBRATION`). Worker count, APA window, MSD probes and fits, and how to map simulation units → physical units — all from the `analysis:` block.

Helper functions you'll see called elsewhere:

- `get_condition(name)` — look up one entry in `SIMULATION_CONDITIONS`.
- `list_conditions()` — list all condition names.
- `get_ctcf_arrays(ctcf_type)` — returns `(positions, orientations)` for the untiled locus; raises if that sample's BED has not been made.
- `get_tiled_ctcf_arrays(ctcf_type)` — same but repeated across the tiled polymer (what the simulations actually use).
- `load_ctcf_from_bed(path)` — read a BED6 file and return `(positions, orientations)` as monomer-index arrays.

2. Hi-C data preparation

In ChromSimPipe, Hi-C conversion is handled automatically by the `convertHic` Snakemake rule: it calls `hic2cool convert` on each `.hic` file in the samplesheet's `Hi-C` column to produce a single-resolution `output/mcool/<sample>.cool` at the configured resolution (default 1 kb).

The Bonev 2017 mESC/cortical-neuron download scripts and the Sox2 quick-look plotter from the original lab project now live in `legacy/` (see `legacy/README.md`); they are not part of the workflow.

3. CTCF annotation

`scripts/extract_ctcf_sites.py`

Purpose. Given a CTCF ChIP-seq or CUT&Tag peak file, a genome FASTA, and a CTCF motif (MEME format), run FIMO to assign strand orientation to each peak in the locus, and write an oriented BED6 file.

The `orientCTCF` Snakemake rule calls this for every sample a condition uses as its `ctcf_type`; every argument comes from the config. To run manually (inside the `ctcf` environment):

```
        python scripts/extract_ctcf_sites.py \
            --bed /path/to/peaks.narrowPeak \
            --region chr1:64210000-66170000 \
            --genome /path/to/genome.fa \
            --motif output/motifs/MA0139.1.meme \
            --assembly hg38 \
            --fimo-thresh 1e-3 --flank 100 --unoriented drop --
resolution 1000 \
            --output
output/ctcf/ctcf_oriented_hg38_condition1_chr1_64210000_66170000.bed
```

Output: BED6 — one row per peak with a FIMO hit; column 6 (+/−) is the motif strand.

`scripts/validate_ctcf.py`

Purpose. Pre-flight sanity check before any simulation. Loads the oriented CTCF BEDs, converts genomic coordinates to monomer indices, and prints a report. Called by the `validateCTCF` Snakemake rule; also runnable standalone (inside the `sim` environment):

```
CHROMSIM_CONFIG=config/ChromSimConfig.yaml python scripts/validate_ctcf.py
```

Checks: BED parses cleanly; all sites are inside the simulation window; coordinate → monomer-index mapping is monotone; prints counts per condition and sites gained / lost / shared. Exits non-zero on failure, including when a BED is missing.

`scripts/export_conserved_ctcf_bed.py`

Purpose. Intersect two oriented CTCF BEDs, keep only sites shared in both (same position and orientation), and write a conserved-sites BED. Useful for testing "CTCF binding didn't change, only cohesin did" scenarios.

4. The LEF dynamics engine

`scripts/lef_dynamics.py`

Purpose. A lightweight 1D Python simulator for loop-extruding factors (cohesins). No 3D physics — just: where are the cohesins on the lattice, where are they stalled, where do they jump when they unbind.

Class: `LEFSimulator`. Constructor arguments:

- `N` — lattice length in monomers.
- `n_lefs` — how many cohesins to place.
- `lifetime` — mean number of steps a cohesin stays bound before unbinding (`unbind probability per step = 1/lifetime`).
- `ctcf_positions`, `ctcf_orientations` — from `get_ctcf_arrays`.
- `ctcf_capture` — probability of stalling when a cohesin arm moves onto a CTCF site with the blocking orientation.
- `ctcf_release` — probability per step that a stalled cohesin lets go.
- `rng_seed` — for reproducibility.

What each `step()` call does.

1. Rebuild the occupancy grid from every cohesin's left / right arm positions (so collisions are up-to-date if arms moved last step).
2. For each cohesin (in a random permutation, so no directional bias):
   - Age by one step.
   - With probability `1/lifetime`, unbind → clear occupancy, call `_place_lef()` to re-seed at a random free pair of positions.
   - Left arm (moves leftward): if the next position is free and not a blocking CTCF site (or we rolled `> ctcf_capture`), move there. If it is blocking and we rolled `< ctcf_capture`, flag as stalled.

    - Right arm (moves rightward): mirror logic.
    - If already stalled on CTCF, roll `< ctcf_release` to escape.

Public methods you'll see used.

- `get_bonds()` — list of `(left, right)` tuples of current cohesin-bridged monomer pairs. Fed to polychrom as harmonic SMC bonds.
- `get_bond_arrays()` — same but as two parallel NumPy arrays.
- `get_loop_sizes()` — current loop lengths (in monomers).
- `run(n_steps)` — convenience: step `n_steps` times, return the list of per-step bond lists.

Output variables you should know. After a `step()`, these arrays reflect the current state: `left_pos[i]`, `right_pos[i]`, `ages[i]`, `left_stalled[i]`, `right_stalled[i]`, `occupied[pos]`.

5. Running simulations

5a. `scripts/run_simulation_shard.py` — the Snakemake entry point

This is what every `simulateShard` SLURM job calls (36 jobs with the example config: 3 conditions × 3 replicates × 4 shards).

Arguments:

Flag	Meaning
`--condition NAME`	One of the names in `SIMULATION_CONDITIONS`. Sets cohesin params + CTCF track.
`--replicate N`	Integer replicate index (drives RNG seed).
`--shard-index I`	Which shard of this replicate (0…`n_shards-1`).
`--n-shards K`	Total shards per replicate (default 4).
`--gpu I`	CUDA device index.
`--output DIR`	Output base directory (the workflow passes `output/sims`).
`--shard-index-offset M`	Add `M` to shard index — use to extend existing runs without overwriting.

Shard output goes to `<output>/<params_name>_ctcf-<ctcf_type>_rep<r>_shard<i>/`.

What it does:

1. Look up the condition → resolve to `(params_dict, ctcf_type)`. Check that the engine in `sim_run.engine` can be imported; stop if not (no silent fallback to another engine).
2. Build the tiled polymer (`N = TILING["chrom_size"]`), fetch tiled CTCF arrays.
3. Warm up `LEFSimulator` for `warmup_blocks` steps (throw away).
4. **Pre-compute** all LEF bond states for every block in advance — the 1D dynamics are independent of 3D physics, so doing this up front lets the polychrom loop replay bonds from a list.
5. Register all unique bond pairs as `harmonic_bonds` at `k=0`; flip `k` each block (~10× faster than adding/removing bonds).
6. `sim.local_energy_minimization()` relaxes the starting configuration.
7. Run `total_blocks / n_shards` blocks; `HDF5Reporter` saves the conformation of every block (`hdf5_frames_per_file` per file). `save_every` and the 3D side of `warmup_blocks` are not applied in this runner.

5b. `scripts/merge_shards.py`

Concatenates per-shard HDF5 streams into `merged_<condition>_rep<N>/` with the same layout the analysis expects. Writes `merge_metadata.json` recording which shard IDs contributed.

```
--condition, --replicate, --n-shards, --results-dir
```

5c. `scripts/run_simulation.py` — single-run / local testing

One-shot simulation without sharding. Useful for local testing or LEF-only sanity checks:

```
        python scripts/run_simulation.py \
            --condition cond1_baseline --replicate 0 --gpu 0 --
output output/sims
```

`--engine` defaults to `sim_run.engine`; `--engine lef_only` skips the 3D polymer entirely and writes only a 2D contact map from 1D LEF statistics.

6. Post-simulation analysis

6a. The orchestrator: `scripts/run_analysis_all.py`

One command that walks every `merged_<condition>_rep<N>` directory, runs the full analysis suite, and writes per-condition outputs. Called by the `analyzeCondition` Snakemake rule.

How to run directly (e.g. for one condition):

```
        CHROMSIM_CONFIG=config/ChromSimConfig.yaml python
scripts/run_analysis_all.py \
            --results-dir output/sims \
            --output-dir output/analysis/cond1_baseline \
            --condition cond1_baseline \
            --hic-dir output/mcool \
            --n-jobs 4 --skip-existing --no-msd
```

Selected flags:

Flag	What it does
`--results-dir DIR`	Where to find merged simulation directories.
`--output-dir DIR`	Where to write outputs.
`--condition NAME`	Analyse one condition only.
`--n-jobs N`	Parallel workers for contact detection (default `analysis.n_jobs`).
`--skip-existing`	Skip dirs that already have `sim_contact_map.npy`.
`--no-msd`	Skip MSD computation (the Snakemake rule adds this unless `analysis.msd: True`).
`--hic-dir DIR`	Where to find `<compare_hic>.cool` for each condition (falls back to `output/mcool/`).
`--mcool TYPE=PATH`	Override the `.cool` for one Hi-C type (repeatable).
`--ctcf-bed TYPE=PATH`	Override the oriented CTCF BED for one ctcf_type (repeatable).

Per-replicate outputs (in `<sim_dir>/analysis/`):

File	What it is
`sim_contact_map.npy`	2D contact matrix from 3D conformations
`sim_ps_curve.npz`	Contact probability vs genomic distance
`sim_insulation.npy`	Insulation score along the locus

File	What it is
`apa_convergent.png`	Aggregate Peak Analysis at convergent CTCF pairs
`sim_vs_exp_map.png`	Side-by-side sim vs experimental Hi-C heatmap
`comparison_metrics.json`	Pearson + stratum-adjusted correlations with experimental Hi-C
`msd_<label>.npz`	Two-point MSD curves (if `--no-msd` not set)
`calibration.json`	Physical-unit mapping: nm/monomer, sec/frame

6b. Analysis modules (`analysis/`)

Module	Purpose
`contact_maps.py`	Contact matrix from conformations via `scipy.spatial.cKDTree`; also P(s), insulation, map-vs-map correlation
`ps_curve.py`	Power-law fitting of P(s)
`absolute_quant.py`	Aggregate Peak Analysis (APA) at convergent CTCF pairs
`ctcf_plotting.py`	Contact-map heatmaps with CTCF arrow strips
`experimental_compare.py`	Sim vs experimental Hi-C comparison (`.cool` and `.npy`)
`msd_two_point.py`	Mean-squared displacement of monomer pairs; sub-diffusive exponent fit
`polymer_dynamics.py`	Radius of gyration, loop fractions, dwell-time distributions
`calibration.py`	Map simulation units → physical units (nm/monomer, sec/frame)
`batch_analysis.py`	Older single-pass analysis over all result dirs (simpler than the orchestrator)

6c. `scripts/summarize_analysis.py`

Collects scalar metrics from all conditions into one CSV/XLSX table:

```
python scripts/summarize_analysis.py \
    --analysis-dir output/analysis \
```

```
    --reference-condition cond1_baseline
```

6d. `scripts/check_results.py`

Lists every directory under `output/sims`, its frame count, and whether analysis outputs are present. Quick inventory after a SLURM batch.

Legacy cluster scripts (`legacy/cluster/`)

The pre-Snakemake SLURM scripts were written for the lab's original cluster (mambaforge under `ric.tiget`, `cuda` / `workq` partitions) and the mESC/neuron study, so they have been moved to `legacy/cluster/` and are not maintained. **Use `sbatch ChromSimPipe.sh`**, which handles the full DAG on Longleaf.

`afterok` vs `afterany`

- `--dependency=afterok:<JID>` — fire only if upstream finished with exit 0. Use between pipeline stages where failure upstream makes the next stage meaningless.
- `--dependency=afterany:<JID>` — fire regardless of how upstream ended. Use for resume-chain links meant to recover from wall-time TIMEOUTs. `submit_analysis_resume_chain.sh` uses this pattern.

Conventions you'll see everywhere

- **Monomer = `resolution` bp (1 kb by default).** Monomer index `i` covers `REGION_START + i * RESOLUTION` to `REGION_START + (i+1) * RESOLUTION`.
- **Tiled chromosome.** Simulations run on `chrom_size` (70,000) monomers = 28 copies of the locus with padding. Contact maps are averaged back to `N_MONOMERS` (1,960) for comparison. This gives ~28× more contact statistics at no extra physics cost.
- **Output directory names.** Shards: `output/sims/<params_name>_ctcf-<ctcf_type>_rep<N>_shard<S>/`. Merged: `output/sims/merged_<params_name>_ctcf-<ctcf_type>_rep<N>/`. The `_ctcf-<type>` part separates runs with the same cohesin params but different CTCF tracks.
- **RNG seed.** `seed_base + replicate * 10000 + shard * 1000` (`seed_base` = 42 in the config). Always derived from indices so reruns are reproducible.
- **Every run writes `params.json`** with the full parameter dict + CTCF type + replicate. If a result looks wrong, check `params.json` first.

- **Completion markers.** Snakemake tracks simulation, merge, and analysis steps through `.done` files in `output/done/`. Delete the relevant `.done` file if you need to re-run a step (for example after changing a parameter that only the scripts read).

Common pitfalls

CUDA architecture error. `openmm.OpenMMException: nvrtc: error: invalid value for --gpu-architecture (-arch)` means the environment's CUDA cannot compile for the allocated GPU. CUDA 13 dropped Pascal (GTX 10xx) and Volta (V100); `envs/cohesin_sim.yml` pins `cuda-version=12` for this reason. Check with `conda list -p <env> | grep cuda-version` and rebuild if it says 13.

No Hi-C comparison in the analysis output. The analysis catches errors in each step, logs a warning and continues, so check `output/logs/analyzeCondition_*.err` for `WARNING`. A missing normalization column (`no 'KR' normalization`) is fixed with `analysis.hic_balance`.

Snakemake lock error after a failed run. Run `bash unlock.sh`, then resubmit.

`ModuleNotFoundError: configs` or `analysis`. Every script in `scripts/` inserts the repo root onto `sys.path`. Run from the repo root, or set `PYTHONPATH=/path/to/ChromSimPipe`.

`Samplesheet ... has no 'CTCF_Peaks' column.` `ChromSimSamplesheet.txt` must be tab-separated and include the headers `CTCF_Peaks` and `Hi-C`, plus every column named in `fileNamesFrom`.

`hic2cool` fails. Check the sample's `Hi-C` path in the samplesheet, and `output/logs/convertHic_<sample>.err`.

Analysis fails with "No results found". Check that `output/sims/merged_*_rep*/` directories exist. If shards completed but the merged directory is missing, delete `output/done/mergeShards_<condition>_rep<N>.done` and resubmit.

No emails. Check `$CHROMSIM_EMAIL` is set in the shell you submit from (sbatch passes it on), and that `echo test | mail -s test $CHROMSIM_EMAIL` works on the cluster; otherwise set `notify: command:` to what does.

`Conda must be version 24.7.1 or later.` The cluster's conda is older than Snakemake 8.27 accepts (true of Longleaf's and Pioneer's modules). Install Miniforge once (`bash Miniforge3-Linux-x86_64.sh -b -p $HOME/miniforge3`), delete the launcher's `env/` folder, and rerun; the shipped `env.sh` files activate it.

`Cannot load module ... because these module(s) are loaded.` Some clusters (Longleaf among them) refuse to load a `python` module and an `anaconda` module together. Load only the conda distribution in that site's `env.sh`; it brings its own Python.

Where the logs are. Each job's stdout / stderr: `output/logs/<rule>_<wildcards>.out|.err`. SLURM's own job logs: `output/logs_slurm/`. The launcher (main Snakemake process): `ChromSimPipe-<jobid>.out` in the repo root.